

Actúa como asistente de programación para una práctica de Estructura de Datos en Python. ^

Contexto:

Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran resultados, así como la clase que realiza la carga y la detección del evento para convertir la expression.

Objetivo:

Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase que contenga una pila y un método para llevar acabo la conversion.

¡Hola! Claro que sí, con gusto te ayudo a integrar esta funcionalidad en tu práctica de Estructuras de Datos.

Como ya tienes la interfaz en Qt Designer y la clase `Stack` base, nos enfocaremos exclusivamente en crear la clase convertidora (**el modelo lógico**). Esta clase recibirá la expresión infija, utilizará tu pila para procesar los operadores según su precedencia y devolverá la expresión posfija (PostFix).

Aquí tienes la estructura de la clase que necesitas implementar.

Estructura de la Clase `InfixToPostfix`

Vamos a crear una clase llamada `InfixToPostfixConverter`. Asumiré que tu clase `Stack` ya tiene los métodos clásicos: `push()`, `pop()`, `peek()` (o `top()`), `is_empty()` (o `isEmpty()`) y `size()`.

Python

```
# Asegúrate de importar tu clase Stack desde el archivo correspondiente
# from tu_archivo_pila import Stack

class InfixToPostfixConverter:
    def __init__(self):
        # Creamos una instancia de la pila que desarrollaron en clase
        self.stack = Stack()

        # Definimos la precedencia de los operadores utilizando un
        # diccionario
        # A mayor número, mayor prioridad
        self.precedence = {
            '+': 1,
            '-': 1,
            '*': 2,
            '/': 2,
            '^': 3
        }

    def _get_precedence(self, operator):
        """Método auxiliar para obtener la prioridad de un operador."""
        return self.precedence.get(operator, 0)

    def convert(self, infix_expression: str) -> str:
        """
        Método principal que realiza la conversión de Infija a Posfija.
        Recibe un string (ej. "A+B*C") y devuelve un string (ej. "ABC*+")
        """
        postfix_result = []

        # Limpiamos espacios en blanco por si el usuario introdujo la
        # expresión con espacios
        # Nota: Este algoritmo asume tokens individuales (letras o
        # números de un dígito).
        tokens = infix_expression.replace(" ", "")

        for token in tokens:
            # Caso 1: Si el token es un operando (letra o número), se
            # añade directamente al resultado
            if token.isalnum():
                postfix_result.append(token)

            # Caso 2: Si es un paréntesis de apertura '(', se empuja a la
            # pila
            elif token == '(':
                self.stack.push(token)

            # Caso 3: Si es un paréntesis de cierre ')':
            elif token == ')':
                # Sacamos elementos de la pila y los agregamos al
                # resultado hasta encontrar el '('
                while not self.stack.isEmpty():
                    postfix_result.append(self.stack.pop())
```

```

        while not self.stack.is_empty() and self.stack.peek() !=
'(':
            postfix_result.append(self.stack.pop())

            # Eliminamos el paréntesis de apertura de la pila
            if not self.stack.is_empty() and self.stack.peek() ==
'(':
                self.stack.pop()

            # Caso 4: Si el token es un operador (+, -, *, /, ^)
            else:
                # Mientras la pila no esté vacía y el operador en el tope
tenga
                # mayor o igual precedencia que el operador actual...
                while (not self.stack.is_empty() and
                    self._get_precedence(self.stack.peek()) >=
self._get_precedence(token)):
                    postfix_result.append(self.stack.pop())

                # Finalmente, empujamos el operador actual a la pila
                self.stack.push(token)

            # Caso 5: Al terminar de leer la expresión, vaciamos los
operadores restantes en la pila
            while not self.stack.is_empty():
                postfix_result.append(self.stack.pop())

        # Unimos la lista en un solo string para retornar el resultado
final
        return "".join(postfix_result)

```

dime donde debo agregar cada cosa

Para que todo quede perfectamente organizado y no rompas la estructura que ya te dio tu profesor en clase, lo ideal es seguir el patrón de diseño **MVC (Modelo-Vista-Controlador)** que se suele usar en interfaces gráficas.

Aquí tienes el mapa exacto de dónde debes colocar cada pieza de código en tu proyecto:

1. El Modelo (La Lógica)

¿Dónde va? En el archivo donde tengas tu clase `Stack` (por ejemplo, `estructuras.py` o `pila.py`), o bien en un archivo nuevo llamado `conversor.py`.

Aquí vas a colocar la clase `InfixToPostfixConverter` que creamos arriba.

Regla de oro: Este archivo **no** debe saber nada de Qt, ni de botones, ni de la interfaz gráfica. Solo procesa texto y usa la pila.

Python

```
# conversor.py (o dentro de tu archivo de estructuras)
class InfixToPostfixConverter:
    # ... (Aquí va todo el código que te pasé en la respuesta anterior)
```

2. El Controlador (La Ventana/Evento)

¿Dónde va? En tu archivo principal o en la clase que carga el archivo `.ui` (o la clase heredada de `QMainWindow/QDialog`). Suele llamarse `main.py`, `ventana_principal.py` o `controlador.py`.

Aquí es donde debes hacer tres cosas clave:

1. **Importar** la nueva clase conversora al principio del archivo.
2. **Crear el método** que reaccione al botón.
3. **Conectar** el evento del botón de Qt Designer con tu método.

Mira este ejemplo de cómo debe quedar estructurado ese archivo:

```
Python

import sys
from PyQt5.QtWidgets import QApplication, QMainWindow
from PyQt5.uic import loadUi

# 1. IMPORTAR TU CLASE CONVERSORA
from conversor import InfixToPostfixConverter

class MiVentana(QMainWindow):
    def __init__(self):
        super(MiVentana, self).__init__()
        # Aquí cargas tu archivo de Qt Designer
        loadUi("tu_interfaz.ui", self)

        # 3. CONECTAR EL EVENTO DEL BOTÓN
        # Reemplaza 'btn_convertir' por el objectName que le diste a tu
        # botón en Qt Designer
        self.btn_convertir.clicked.connect(self.procesar_expresion)

    # 2. EL MÉTODO QUE DETECTA EL EVENTO
    def procesar_expresion(self):
        # Reemplaza 'txt_infijo' por el objectName de tu QLineEdit de
        # entrada
```

```

expresion_infija = self.txt_infijo.text()

# Instanciamos el convertidor
convertidor = InfixToPostfixConverter()

# Realizamos la conversión
expresion_posfija = convertidor.convert(expresion_infija)

# Reemplaza 'lbl_resultado' por el objectName de tu QLabel o
QLineEdit de salida
self.lbl_resultado.setText(expresion_posfija)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    ventana = MiVentana()
    ventana.show()
    sys.exit(app.exec_())

```

3. La Vista (Qt Designer)

¿Dónde va? En tu archivo `.ui`. Aquí no tienes que programar código, solo asegurarte de revisar los nombres (`objectName`) en el panel derecho de Qt Designer:

- Selecciona tu cuadro de texto de entrada y mira su `objectName` (ej. `txt_infijo`).
- Selecciona tu botón y mira su `objectName` (ej. `btn_convertir`).
- Selecciona la etiqueta o cuadro de texto de salida y mira su `objectName` (ej. `lbl_resultado`).

Esos mismos nombres son los que usarás en el código del **Paso 2** para que Python sepa a qué componente de la pantalla te estás refiriendo.